An Equity Injection Problem

Mathematical Framework and Numerical Methods

Project: Risk Management on Financial Markets

Based on: Bayraktar, Guo, Tang, Zhang (2025)

Group Members:

[Your Names Here]

March 5, 2026

Contents

1 Introduction **3**

2 Problem Formulation **3**
2.1 The Banking Network . . . 3
2.2 State Space . . . 3
2.3 Dynamics and Shocks . . . 3
2.4 Two Scenarios: Independent vs. Interconnected Banks . . . 4
2.4.1 Case 1: Independent Banks . . . 4
2.4.2 Case 2: Interconnected Banks (Default Cascades) . . . 4

3 Optimization Objectives **4**
3.1 Objective U: Maximize Expected Number of Survivors . . . 4
3.2 Objective V: Maximize Probability All Survive . . . 4
3.3 The Optimization Problem . . . 5

4 Dynamic Programming Principle **5**
4.1 Value Function . . . 5
4.2 The Bellman Equation . . . 5
4.3 Proof Sketch . . . 6
4.4 The Expectation in Detail . . . 6

5 Why We Cannot Solve This Exactly **6**
5.1 Curse of Dimensionality . . . 6
5.2 Parametric vs Non-Parametric Approach . . . 7

6 Function Approximation: The Key Idea **7**
6.1 Linear Function Approximation . . . 7
6.2 Feature Functions . . . 7
6.3 Example Features . . . 8
6.4 Why Linear Combinations? . . . 8

7 Regression-Based Dynamic Programming **8**
7.1 The Approximate Bellman Equation . . . 8
7.2 Backward Induction Structure . . . 8

8 The Regression Problem **9**
8.1 Training Data Generation . . . 9
8.2 Target Computation via Monte Carlo . . . 9
8.3 The Regression Objective . . . 9

9 Regularization: Ridge Regression **10**
9.1 Why Regularize? . . . 10
9.2 Ridge Regression . . . 10
9.3 Properties of Ridge Regression . . . 11
9.4 Choosing λ: Cross-Validation . . . 11

10 The Complete Algorithm **12**
10.1 Step 0: Initialization at Terminal Time . . . 12
10.2 Backward Iteration . . . 13

11 Error Analysis and Convergence **13**
11.1 Sources of Error . . . 13
11.2 Bias-Variance Trade-off . . . 14

12 Mathematical Subtleties **14**
12.1 Default Cascades (Case 2) . . . 14
12.2 Bellman Error . . . 15

13 Theoretical Justification **15**
13.1 Fitted Value Iteration . . . 15
13.2 Connection to Reinforcement Learning . . . 16

14 Summary: The Mathematical Flow **16**

15 Practical Implementation Tips **16**
15.1 Feature Scaling . . . 16
15.2 Computational Complexity . . . 17
15.3 Numerical Linear Algebra . . . 17

16 Key References **17**

A Quick Reference: Key Formulas **17**
A.1 Bellman Equation . . . 17
A.2 Function Approximation . . . 17
A.3 Ridge Regression . . . 17
A.4 Monte Carlo Estimator . . . 18

1 Introduction

This document provides a comprehensive mathematical treatment of the optimal equity injection problem for banking networks. We consider a regulator with limited budget who must optimally allocate capital injections to maximize financial system stability.

Project Overview

Main Question: How should a central bank optimally distribute a limited bailout budget across failing banks over time to maximize the stability of the financial system?

2 Problem Formulation

2.1 The Banking Network

Consider a homogeneous banking network consisting of N banks, labeled $i = 1, 2, \ldots, N$. Time is discrete: $t \in \{0, 1, 2, \ldots, T\}$.

Definition 2.1 (Bank Capital)**.** Each bank i has a capital level $S_t^i \in \mathbb{R}$ at time t:

- $S_t^i > 0$: Bank i is solvent
- $S_t^i \leq 0$: Bank i defaults

Definition 2.2 (Survival Indicator)**.** We define the survival indicator:

$$e_t^i = \begin{cases} 1 & \text{if bank } i \text{ is alive at time } t \\ 0 & \text{if bank } i \text{ has defaulted by time } t \end{cases}$$

2.2 State Space

Definition 2.3 (State Space)**.** The state of the system at time t is described by:

$$(S_t, e_t) = \big((S_t^1, \ldots, S_t^N), (e_t^1, \ldots, e_t^N)\big) \in \mathbb{R}^N \times \{0, 1\}^N$$

The combined state space is:

$$\mathcal{S} = \mathbb{R}^N \times \{0, 1\}^N$$

This is a **hybrid space** – part continuous, part discrete.

2.3 Dynamics and Shocks

Definition 2.4 (Random Shocks)**.** Each period, bank i experiences a random profit or loss X_{t+1}^i:

- $X_{t+1}^i > 0$: Profit
- $X_{t+1}^i < 0$: Loss

The shock vector $X_{t+1} = (X_{t+1}^1, \ldots, X_{t+1}^N)$ is adapted to the filtration $(\mathcal{F}_t)_{t=0}^T$.

Definition 2.5 (Regulator's Actions)**.** At each time t, the regulator chooses capital injections:

$$b_t = (b_t^1, \ldots, b_t^N) \in \mathbb{R}_+^N$$

subject to the budget constraint:

$$\sum_{i=1}^N b_t^i \leq c$$

where $c > 0$ is the total available budget.

Definition 2.6 (Capital Dynamics)**.** The capital of bank i evolves as:

$$\boxed{S_{t+1}^i = S_t^i + X_{t+1}^i + b_{t+1}^i} \tag{1}$$

The survival status updates:

$$e_{t+1}^i = e_t^i \cdot \mathbb{1}_{\{S_{t+1}^i > 0\}} \tag{2}$$

where $\odot$ denotes element-wise multiplication.

2.4 Two Scenarios: Independent vs. Interconnected Banks

2.4.1 Case 1: Independent Banks

Banks are isolated – if one fails, others are unaffected. The dynamics are given by equation (1) only.

2.4.2 Case 2: Interconnected Banks (Default Cascades)

Banks have interbank liabilities. When bank i defaults, every surviving bank suffers an immediate loss equal to β/N, where $\beta > 0$ is the total interbank liability.

This can cause a **cascade effect**:

$$S_{t+1}^i = S_t^i + X_{t+1}^i + b_{t+1}^i - \frac{\beta}{N} \cdot D_t \tag{3}$$

where D_t is the number of banks that default at time t.

3 Optimization Objectives

3.1 Objective U: Maximize Expected Number of Survivors

Definition 3.1 (Objective U)**.**

$$U := \sum_{i=1}^{N} \mathbb{1}_{\{S_t^i > 0,\, \forall t \in \{1,\ldots,T\}\}}$$

Goal: Maximize $\mathbb{E}[U]$ = expected number of banks surviving to time T.

3.2 Objective V: Maximize Probability All Survive

Definition 3.2 (Objective V)**.**

$$V := \prod_{i=1}^{N} \mathbb{1}_{\{S_t^i > 0,\, \forall t \in \{1,\ldots,T\}\}}$$

Goal: Maximize $\mathbb{E}[V] = \mathbb{P}(\text{all banks survive to time } T)$.

3.3 The Optimization Problem

Optimization Problem

Find the optimal control policy $(b_1, \ldots, b_T)$ that solves:

$$\sup_{(b_1,\ldots,b_T)\in\mathcal{A}} \mathbb{E}[U] \quad \text{or} \quad \sup_{(b_1,\ldots,b_T)\in\mathcal{A}} \mathbb{E}[V]$$

subject to:

- Dynamics: $S_{t+1}^i = S_t^i + X_{t+1}^i + b_{t+1}^i$
- Budget: $\sum_{i=1}^N b_t^i \leq c$
- Non-negativity: $b_t^i \geq 0$
- Predictability: b_t is $\mathcal{F}_{t-1}$-measurable

4 Dynamic Programming Principle

4.1 Value Function

Definition 4.1 (Value Function)**.** For a given objective (U or V), define:

$$V_t(x,e) := \sup_{(b_t,b_{t+1},\ldots,b_T)\in\mathcal{A}_t} \mathbb{E}_t\left[\text{Payoff at time } T \mid S_t = x, e_t = e\right]$$

where $\mathcal{A}_t$ is the set of admissible controls from time t onward.

For objective U:

$$V_t^U(x,e) = \sup_{(b_s)_{s=t}^T\in\mathcal{A}_t} \mathbb{E}_t\left[\sum_{i=1}^N \mathbb{1}_{\{S_T^i>0,\, e_T^i=1\}} \mid S_t = x, e_t = e\right]$$

For objective V:

$$V_t^V(x,e) = \sup_{(b_s)_{s=t}^T\in\mathcal{A}_t} \mathbb{E}_t\left[\prod_{i=1}^N \mathbb{1}_{\{S_T^i>0,\, e_T^i=1\}} \mid S_t = x, e_t = e\right]$$

4.2 The Bellman Equation

Theorem 4.1 (Dynamic Programming Principle)**.** The value function satisfies the Bellman equation:

$$\boxed{V_t(x,e) = \sup_{a\in\mathcal{A}(x,e)} \mathbb{E}\left[V_{t+1}(S_{t+1}, e_{t+1}) \mid S_t = x, e_t = e, b_t = a\right]} \tag{4}$$

with dynamics:

$$S_{t+1} = x + X_{t+1} + a \tag{5}$$

$$e_{t+1} = e \odot \mathbb{1}_{\{S_{t+1}>0\}} \tag{6}$$

Theorem 4.2 (Terminal Condition)**.**

$$V_T(x,e) = \begin{cases} \sum_{i=1}^N e_i & \text{for objective U} \\ \prod_{i=1}^N e_i & \text{for objective V} \end{cases}$$

4.3 Proof Sketch

Sketch. 1. By the tower property of conditional expectation:

$$\mathbb{E}_t[\text{Payoff}] = \mathbb{E}_t\left[\mathbb{E}_{t+1}[\text{Payoff}]\right]$$

2. The supremum over all future controls decomposes:

$$\sup_{(b_t, b_{t+1}, \ldots, b_T)} = \sup_{b_t} \sup_{(b_{t+1}, \ldots, b_T)}$$

3. The inner supremum is precisely $V_{t+1}(S_{t+1}, e_{t+1})$ by definition.

4. Therefore:

$$V_t(x, e) = \sup_{b_t} \mathbb{E}_t\left[V_{t+1}(S_{t+1}, e_{t+1})\right]$$

This is Bellman's principle of optimality. □

4.4 The Expectation in Detail

Given state (x, e) and action a:

$$\mathbb{E}\left[V_{t+1}(S_{t+1}, e_{t+1}) \mid S_t = x, e_t = e, b_t = a\right] \tag{7}$$

$$= \int_{\mathbb{R}^N} V_{t+1}\left(x + \xi + a,\ e \odot \mathbb{1}_{\{x+\xi+a>0\}}\right)\, dF_{X_{t+1}}(\xi) \tag{8}$$

where $F_{X_{t+1}}$ is the distribution function of the shock vector.

Remark 4.1. This integral:

- Is over $\mathbb{R}^N$ (high-dimensional)
- Has indicator functions (non-smooth)
- Requires knowing V_{t+1} everywhere
- May not have closed form

Solution: Monte Carlo approximation!

5 Why We Cannot Solve This Exactly

5.1 Curse of Dimensionality

The Fundamental Problem

The state space $\mathcal{S} = \mathbb{R}^N \times \{0, 1\}^N$ is **uncountably infinite**.
To solve numerically via grid methods:

- Discretize each S_t^i into K points
- Total grid points: $K^N \times 2^N$

Example 5.1 (Computational Infeasibility). For $N = 10$ banks, $K = 100$ grid points per dimension:

$$\text{Grid size} = 100^{10} \times 2^{10} \tag{9}$$

$$= 10^{20} \times 1024 \tag{10}$$

$$\approx 10^{23} \text{ states} \tag{11}$$

Storage requirement: 8×10^{23} bytes $= 8 \times 10^{11}$ TB

This is impossible!

5.2 Parametric vs Non-Parametric Approach

Approach	Advantages	Disadvantages
Grid-based (Non-parametric)	Exact (up to discretization) No approximation error	Exponential storage Curse of dimensionality
Function Approx. (Parametric)	Compact representation Scalable to large N	Approximation error Need good features

Table 1: Comparison of solution methods

6 Function Approximation: The Key Idea

6.1 Linear Function Approximation

Definition 6.1 (Parametric Value Function)**.** Approximate the value function as:

$$V_t(x, e) \approx V_t^\theta(x, e) := \sum_{p=1}^{P} \theta_p \cdot \phi_p(x, e) = \langle \theta, \phi(x, e) \rangle \tag{12}$$

where:

- $\phi(x, e) = (\phi_1(x, e), \ldots, \phi_P(x, e))^T \in \mathbb{R}^P$: **Feature vector**
- $\theta = (\theta_1, \ldots, \theta_P)^T \in \mathbb{R}^P$: **Weight vector**
- $P \ll K^N$: Number of features

Key insight: Instead of storing values at exponentially many states, we store only P parameters!

6.2 Feature Functions

Definition 6.2 (Good Features)**.** Feature functions $\{\phi_p\}_{p=1}^P$ should:

1. **Capture relevant information:** Depend on aggregates like $\sum x_i$, $\min x_i$, $\sum e_i$
2. **Exploit symmetry:** If banks are homogeneous, V_t should be permutation invariant
3. **Be linearly independent:** Ensure $\Phi^T \Phi$ is invertible
4. **Cover different regimes:** Linear, nonlinear, and interaction terms

6.3 Example Features

Example 6.1 (Typical Feature Functions).

$$\phi_1(x,e) = \frac{1}{N}\sum_{i=1}^{N} e_i \qquad \text{(fraction of banks alive)} \tag{13}$$

$$\phi_2(x,e) = \frac{1}{N}\sum_{i=1}^{N} x_i \qquad \text{(average capital)} \tag{14}$$

$$\phi_3(x,e) = \frac{\sum_{i=1}^{N} e_i \cdot x_i}{\sum_{i=1}^{N} e_i} \qquad \text{(average capital among survivors)} \tag{15}$$

$$\phi_4(x,e) = \min_{i:e_i=1} x_i \qquad \text{(weakest bank's capital)} \tag{16}$$

$$\phi_5(x,e) = \left(\frac{1}{N}\sum_{i=1}^{N} x_i\right)^2 \qquad \text{(quadratic term)} \tag{17}$$

$$\phi_6(x,e) = \frac{1}{N}\sum_{i=1}^{N}(x_i - \bar{x})^2 \qquad \text{(variance)} \tag{18}$$

$$\phi_P(x,e) = 1 \qquad \text{(constant/intercept)} \tag{19}$$

6.4 Why Linear Combinations?

Proposition 6.1. Linear approximation leads to **convex optimization** in θ:

$$\min_{\theta} \sum_{n=1}^{N} \left(Y^{(n)} - \langle \theta, \phi(x^{(n)}, e^{(n)})\rangle\right)^2$$

This is a **quadratic program** with closed-form solution!

Theorem 6.1 (Universal Approximation)**.** With enough features (polynomials, RBF, etc.), linear combinations can approximate any continuous function arbitrarily well (Stone-Weierstrass theorem).

7 Regression-Based Dynamic Programming

7.1 The Approximate Bellman Equation

Replace exact V_t with approximate V_t^{θ}:

$$V_t^{\theta_t}(x,e) = \sup_{a \in \mathcal{A}(x,e)} \mathbb{E}\left[V_{t+1}^{\theta_{t+1}}(x + X_{t+1} + a, e_{t+1})\right] \tag{20}$$

Goal: Find $\theta_0, \theta_1, \ldots, \theta_T$ such that this equation holds approximately.

7.2 Backward Induction Structure

We work **backwards in time**:

1. **At time T:** Find θ_T such that:

$$\langle \theta_T, \phi(x,e)\rangle \approx \begin{cases} \sum e_i & \text{for U} \\ \prod e_i & \text{for V} \end{cases}$$

2. **At time $t < T$:** Given θ_{t+1} (already computed), find θ_t such that:

$$\langle \theta_t, \phi(x,e)\rangle \approx \sup_a \mathbb{E}\left[\langle \theta_{t+1}, \phi(x + X_{t+1} + a, e_{t+1})\rangle\right]$$

8 The Regression Problem

8.1 Training Data Generation

Definition 8.1 (Training Set Ω_t)**.** Sample representative states that are likely to be visited at time t:

Method:

1. Sample initial states $(x_0^{(n)}, e_0^{(n)})$ from distribution $\mathcal{D}_0$
2. Simulate forward to time t using rollout policy π_{rollout}
3. Record state reached: $(x_t^{(n)}, e_t^{(n)})$

Result: $\Omega_t = \{(x^{(1)}, e^{(1)}), \ldots, (x^{(N_{\text{train}})}, e^{(N_{\text{train}})})\}$

8.2 Target Computation via Monte Carlo

For each state $(x^{(n)}, e^{(n)}) \in \Omega_t$:

Algorithm 1 Target Computation

```
Sample M shock scenarios: ξ^(n,1), ..., ξ^(n,M) ~^iid F_X
for each action a ∈ A_discrete do
    Q_a ← 0
    for m = 1 to M do
        S^(m) ← x^(n) + ξ^(n,m) + a
        e^(m) ← e^(n) ⊙ 𝟙_{S^(m)>0}
        φ^(m) ← φ(S^(m), e^(m))
        Q_a ← Q_a + ⟨θ_{t+1}, φ^(m)⟩
    end for
    Q_a ← Q_a / M
end for
Y^(n) ← max_a Q_a
```

Definition 8.2 (Monte Carlo Estimator)**.**

$$\widehat{\mathbb{E}}[Z] := \frac{1}{M} \sum_{m=1}^{M} Z^{(m)}$$

By the Law of Large Numbers: $\widehat{\mathbb{E}}[Z] \to \mathbb{E}[Z]$ as $M \to \infty$.

8.3 The Regression Objective

We want $V_t^\theta(x, e) = \langle \theta, \phi(x, e) \rangle$ to match the targets:

$$V_t^\theta(x^{(n)}, e^{(n)}) \approx Y^{(n)}, \quad n = 1, \ldots, N_{\text{train}}$$

Definition 8.3 (Least Squares Formulation)**.**

$$\min_{\theta \in \mathbb{R}^P} \sum_{n=1}^{N_{\text{train}}} \left(Y^{(n)} - \langle \theta, \phi(x^{(n)}, e^{(n)}) \rangle \right)^2 \tag{21}$$

In matrix notation:

$$\min_{\theta \in \mathbb{R}^P} \|\mathbf{Y} - \mathbf{\Phi}\theta\|^2 \tag{22}$$

where:

- $\mathbf{Y} = (Y^{(1)}, \ldots, Y^{(N_{\text{train}})})^T \in \mathbb{R}^{N_{\text{train}}}$
- $\mathbf{\Phi} \in \mathbb{R}^{N_{\text{train}} \times P}$ with $\Phi_{n,p} = \phi_p(x^{(n)}, e^{(n)})$

Theorem 8.1 (OLS Solution)**.** The minimizer is:

$$\theta_t = (\mathbf{\Phi}^T \mathbf{\Phi})^{-1} \mathbf{\Phi}^T \mathbf{Y}$$

provided $\mathbf{\Phi}^T \mathbf{\Phi}$ is invertible.

9 Regularization: Ridge Regression

9.1 Why Regularize?

Problems with OLS

Problem 1: Overfitting

- If P is large relative to N_{train}, OLS overfits
- Fits training data perfectly but generalizes poorly

Problem 2: Ill-conditioning

- If features are correlated, $\mathbf{\Phi}^T \mathbf{\Phi}$ is near-singular
- Small changes in $\mathbf{Y}$ cause large changes in θ

9.2 Ridge Regression

Definition 9.1 (Ridge Objective)**.**

$$\min_{\theta \in \mathbb{R}^P} \left\{ \|\mathbf{Y} - \mathbf{\Phi}\theta\|^2 + \lambda \|\theta\|^2 \right\} \tag{23}$$

where:

- $\lambda > 0$: Regularization parameter (hyperparameter)
- $\|\theta\|^2 = \sum_{p=1}^{P} \theta_p^2$: L2 penalty

Interpretation:

- First term: Fit data (minimize residuals)
- Second term: Keep parameters small (penalize complexity)

Theorem 9.1 (Ridge Solution)**.** The minimizer of the ridge objective is:

$$\boxed{\theta_t^{\text{Ridge}} = (\mathbf{\Phi}^T \mathbf{\Phi} + \lambda \mathbf{I}_P)^{-1} \mathbf{\Phi}^T \mathbf{Y}} \tag{24}$$

Proof. Take gradient with respect to θ:

$$\begin{aligned}
\nabla_\theta \left[\|\mathbf{Y} - \mathbf{\Phi}\theta\|^2 + \lambda \|\theta\|^2 \right] &= \nabla_\theta \left[(\mathbf{Y} - \mathbf{\Phi}\theta)^T (\mathbf{Y} - \mathbf{\Phi}\theta) + \lambda \theta^T \theta \right] && (25) \\
&= \nabla_\theta \left[\mathbf{Y}^T \mathbf{Y} - 2\mathbf{Y}^T \mathbf{\Phi}\theta + \theta^T \mathbf{\Phi}^T \mathbf{\Phi}\theta + \lambda \theta^T \theta \right] && (26) \\
&= -2\mathbf{\Phi}^T \mathbf{Y} + 2\mathbf{\Phi}^T \mathbf{\Phi}\theta + 2\lambda\theta && (27)
\end{aligned}$$

Set equal to zero:

$$-\mathbf{\Phi}^T\mathbf{Y} + \mathbf{\Phi}^T\mathbf{\Phi}\theta + \lambda\theta = 0 \tag{28}$$

$$(\mathbf{\Phi}^T\mathbf{\Phi} + \lambda\mathbf{I})\theta = \mathbf{\Phi}^T\mathbf{Y} \tag{29}$$

$$\theta = (\mathbf{\Phi}^T\mathbf{\Phi} + \lambda\mathbf{I})^{-1}\mathbf{\Phi}^T\mathbf{Y} \tag{30}$$

□

9.3 Properties of Ridge Regression

Proposition 9.1 (Ridge Properties).

1. **Always invertible:** $\mathbf{\Phi}^T\mathbf{\Phi} + \lambda\mathbf{I}$ is positive definite for $\lambda > 0$
2. **Shrinkage:** $\|\theta^{\text{Ridge}}\| < \|\theta^{\text{OLS}}\|$
3. **Bias-variance trade-off:**
   - Bias increases with λ
   - Variance decreases with λ
   - Optimal λ minimizes MSE = Bias^2 + Variance

9.4 Choosing λ: Cross-Validation

Algorithm 2 K-Fold Cross-Validation for λ Selection

```
Split Ω_t into K folds: Ω_t = Ω_t^(1) ∪ ··· ∪ Ω_t^(K)
for each λ in candidate set do
  error ← 0
  for k = 1 to K do
    Train on Ω_t \ Ω_t^(k) → get θ^(−k)
    Test on Ω_t^(k) → compute MSE_k
    error ← error + MSE_k
  end for
end for
Choose λ* = arg min_λ error
```

Remark 9.1 (Rule of Thumb). $\lambda \sim 0.01 \times \text{trace}(\mathbf{\Phi}^T\mathbf{\Phi})/P$

10 The Complete Algorithm

10.1 Step 0: Initialization at Terminal Time

Algorithm 3 Initialize θ_T

```
Sample N_init terminal states: (x^(n), e^(n)) ~ D_T
for n = 1 to N_init do
  if objective is U then
    Y^(n) ← Σ_{i=1}^N e_i^(n)
  else if objective is V then
    Y^(n) ← Π_{i=1}^N e_i^(n)
  end if
end for
Build feature matrix: Φ_{n,p} = φ_p(x^(n), e^(n))
Solve: θ_T = (Φ^T Φ + λI)^{-1} Φ^T Y
```

10.2 Backward Iteration

Algorithm 4 Regression-Based Dynamic Programming

for $t = T-1, T-2, \ldots, 1, 0$ **do**
  Step 1: Generate Training States Ω_t
  $\Omega_t \leftarrow \emptyset$
  for $n = 1$ to N_{train} **do**
    Sample initial state: $(x_0^{(n)}, e_0^{(n)}) \sim \mathcal{D}_0$
    $S^{(n)} \leftarrow x_0^{(n)}$, $e^{(n)} \leftarrow e_0^{(n)}$
    for $s = 0$ to $t-1$ **do**
      Sample shock: $X_{s+1}^{(n)} \sim F_X$
      Apply rollout policy: $A_s^{(n)} = \pi_{\text{rollout}}(S^{(n)}, e^{(n)})$
      Update: $S^{(n)} \leftarrow S^{(n)} + X_{s+1}^{(n)} + A_s^{(n)}$
      Update: $e^{(n)} \leftarrow e^{(n)} \odot \mathbb{1}_{\{S^{(n)}>0\}}$
    end for
    $\Omega_t \leftarrow \Omega_t \cup \{(S^{(n)}, e^{(n)})\}$
  end for
  Step 2: Compute Targets via Monte Carlo
  for each $(x^{(n)}, e^{(n)}) \in \Omega_t$ **do**
    Sample M scenarios: $\xi^{(n,1)}, \ldots, \xi^{(n,M)} \overset{\text{iid}}{\sim} F_X$
    $Y^{(n)} \leftarrow -\infty$
    for each action $a \in \mathcal{A}_{\text{discrete}}$ **do**
      $Q_a \leftarrow 0$
      for $m = 1$ to M **do**
        $S^{(m)} \leftarrow x^{(n)} + \xi^{(n,m)} + a$
        $e^{(m)} \leftarrow e^{(n)} \odot \mathbb{1}_{\{S^{(m)}>0\}}$
        $Q_a \leftarrow Q_a + \langle \theta_{t+1}, \phi(S^{(m)}, e^{(m)}) \rangle$
      end for
      $Q_a \leftarrow Q_a / M$
      $Y^{(n)} \leftarrow \max(Y^{(n)}, Q_a)$
    end for
  end for
  Step 3: Ridge Regression
  Build feature matrix: $\Phi_{n,p} = \phi_p(x^{(n)}, e^{(n)})$
  Build target vector: $\mathbf{Y} = (Y^{(1)}, \ldots, Y^{(N_{\text{train}})})^T$
  Solve: $\theta_t = (\mathbf{\Phi}^T \mathbf{\Phi} + \lambda \mathbf{I})^{-1} \mathbf{\Phi}^T \mathbf{Y}$
end for
Output: $\{\theta_0, \theta_1, \ldots, \theta_T\}$

11 Error Analysis and Convergence

11.1 Sources of Error

The approximate value function $V_t^{\theta_t}$ differs from the true V_t due to:

1. **Approximation Error (Function Class):**

$$\epsilon_{\text{approx}} := \min_{\theta \in \mathbb{R}^P} \|V_t - V_t^{\theta}\|$$

   Depends on choice of features; decreases as P increases.

2. **Statistical Error (Finite Sample):**

$$\epsilon_{\text{stat}} \sim O(1/\sqrt{N_{\text{train}}})$$

Due to observing V_t at only N_{train} states.

3. **Monte Carlo Error:**

$$\epsilon_{\text{MC}} \sim O(1/\sqrt{M})$$

Targets $Y^{(n)}$ are noisy due to finite M samples.

4. **Discretization Error (Action Space):**

$$\epsilon_{\text{action}}$$

Depends on action space discretization.

Theorem 11.1 (Error Propagation)**.** Total error propagates backward:

$$\|V_t - V_t^{\theta_t}\| \leq \epsilon_{\text{approx}} + \epsilon_{\text{stat}} + \epsilon_{\text{MC}} + \epsilon_{\text{action}} + \gamma \|V_{t+1} - V_{t+1}^{\theta_{t+1}}\|$$

where $\gamma < 1$ is a contraction factor.

11.2 Bias-Variance Trade-off

Proposition 11.1 (MSE Decomposition)**.** Mean squared error at a test state (x, e):

$$\mathbb{E}\left[(V_t(x,e) - V_t^{\theta_t}(x,e))^2\right] = \underbrace{\text{Bias}^2}_{\text{approximation}} + \underbrace{\text{Variance}}_{\text{estimation}}$$

Method	**Bias**	**Variance**
OLS ($\lambda = 0$)	Low	High
Ridge ($\lambda > 0$)	Slightly higher	Much lower

Table 2: Bias-variance trade-off

Optimal λ minimizes total MSE.

12 Mathematical Subtleties

12.1 Default Cascades (Case 2)

When bank i defaults, survivors lose β/N.

Algorithm 5 Apply Default Cascade

```
Input: (S, e, β, N)
repeat
  e_new ← e ⊙ 𝟙_{S>0}
  D ← Σ_{i=1}^{N} (e_i − e_{new,i})
  if D = 0 then
    break {No new defaults, cascade stops}
  end if
  loss ← β · D/N
  for i = 1 to N do
    if e_{new,i} = 1 then
      S_i ← S_i − loss
    end if
  end for
  e ← e_new
until convergence
Output: (S, e)
```

Remark 12.1. This creates a **fixed-point problem**:

$$e_{t+1} = \Psi(e_t, S_t, X_{t+1}, b_t)$$

where Ψ is the cascade operator.

12.2 Bellman Error

Definition 12.1 (Bellman Error)**.** After computing θ_t, measure:

$$\mathrm{BE}_t := \mathbb{E}_{(x,e)\sim\mathcal{D}}\left[\left(V_t^{\theta_t}(x,e) - \max_a \mathbb{E}[V_{t+1}^{\theta_{t+1}}(\ldots)]\right)^2\right]$$

This quantifies how well the approximate value function satisfies the Bellman equation.

13 Theoretical Justification

13.1 Fitted Value Iteration

Definition 13.1 (Classical Value Iteration)**.** For grid-based methods:

$$V_{k+1}(x,e) = \max_a \mathbb{E}[V_k(x + X + a, e')]$$

Iterate until convergence: $V_k \to V^*$ (optimal value function).

Definition 13.2 (Fitted Value Iteration)**.** Replace exact backup with regression:

1. Start with $V_0^\theta = 0$
2. For $k = 1, 2, \ldots$:
   - Sample states $\{(x^{(n)}, e^{(n)})\}$
   - Compute targets: $Y^{(n)} = \max_a \widehat{\mathbb{E}}[V_{k-1}^\theta(\ldots)]$
   - Fit: $\theta_k = \arg\min_\theta \sum (Y^{(n)} - \langle\theta, \phi(\ldots)\rangle)^2$

Theorem 13.1 (Convergence)**.** Under certain conditions (contraction, bounded approximation error), $V_k^\theta \to V^*$ approximately.

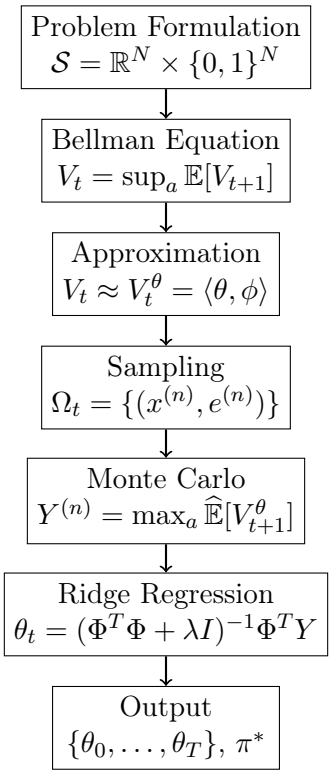
13.2 Connection to Reinforcement Learning

This is **Least Squares Temporal Difference (LSTD)** learning:

- **State:** (S_t, e_t)
- **Action:** b_t
- **Reward:** Terminal payoff at T
- **Transition:** $S_{t+1} = S_t + X_{t+1} + b_t$

We're solving the **Bellman optimality equation** via **function approximation** and **Monte Carlo sampling**.

14 Summary: The Mathematical Flow



15 Practical Implementation Tips

15.1 Feature Scaling

Problem: If features have different scales, regression is ill-conditioned.

Solution: Standardize features:

$$\tilde{\phi}_p = \frac{\phi_p - \mu_p}{\sigma_p}$$

where μ_p, σ_p are mean and std dev of ϕ_p over training data.

15.2 Computational Complexity

Per time step t:

$$\begin{aligned} \text{Trajectory simulation:} \quad & O(N_{\text{train}} \times t) && (31) \\ \text{Monte Carlo:} \quad & O(N_{\text{train}} \times M \times |\mathcal{A}|) && (32) \\ \text{Feature computation:} \quad & O(N_{\text{train}} \times P) && (33) \\ \text{Matrix inversion:} \quad & O(P^3) && (34) \end{aligned}$$

Total: $O(T \times N_{\text{train}} \times M \times |\mathcal{A}| + TP^3)$
Dominant term: Usually Monte Carlo step.
Parallelization: Monte Carlo samples are independent $\Rightarrow$ embarrassingly parallel!

15.3 Numerical Linear Algebra

Method 1: Normal equations

```
theta = np.linalg.solve(Phi.T @ Phi + lambda * np.eye(P),
                        Phi.T @ Y)
```

Method 2: SVD (more stable)

```
U, s, Vt = np.linalg.svd(Phi, full_matrices=False)
theta = Vt.T @ np.diag(s / (s**2 + lambda)) @ U.T @ Y
```

SVD is slower but more numerically stable for ill-conditioned $\mathbf{\Phi}$.

16 Key References

1. Bayraktar, E., Guo, G., Tang, W. and Zhang, Y. (2025). "Systemic robustness: a mean-field particle system approach." *Mathematical Finance.*

2. Longstaff, F. and Schwartz, E. (2001). "Valuing American options by simulation: A simple least-squares approach." *Review of Financial Studies*, 14(1), 113-147.

3. Tsitsiklis, J. and Van Roy, B. (1997). "An analysis of temporal-difference learning with function approximation." *IEEE TAC*, 42(5), 674-690.

4. Bertsekas, D. (2019). *Reinforcement Learning and Optimal Control.* Athena Scientific.

A Quick Reference: Key Formulas

A.1 Bellman Equation

$$V_t(x, e) = \sup_{a \in \mathcal{A}(x,e)} \mathbb{E}\left[V_{t+1}(S_{t+1}, e_{t+1}) \mid S_t = x, e_t = e, b_t = a\right]$$

A.2 Function Approximation

$$V_t(x, e) \approx V_t^{\theta}(x, e) = \sum_{p=1}^{P} \theta_p \phi_p(x, e)$$

A.3 Ridge Regression

$$\theta_t = (\mathbf{\Phi}^T \mathbf{\Phi} + \lambda \mathbf{I})^{-1} \mathbf{\Phi}^T \mathbf{Y}$$

A.4 Monte Carlo Estimator

$$\mathbb{E}[Z] \approx \frac{1}{M} \sum_{m=1}^{M} Z^{(m)}$$